

# Sirius-Skool

Product Requirements Document

Version 1.0 — July 2026

# Sirius-Skool — Product Requirements Document

---

**Version:** 1.0  
**Status:** Draft  
**Date:** July 3, 2026

---

## Table of Contents

---

1. [Executive Summary](#)
  2. [Problem Statement](#)
  3. [Scope](#)
  4. [User Types & Role Hierarchy](#)
  5. [Multi-Tenant Architecture Overview](#)
  6. [Functional Modules](#)
    7. 6.1 Authentication
    8. 6.2 Dashboard
    9. 6.3 Student Management & Admission
    10. 6.4 Attendance Management
    11. 6.5 Result Management
    12. 6.6 Notice Board
    13. 6.7 Notification System
    14. 6.8 Reports
    15. 6.9 Settings
  16. [Cross-Cutting Modules](#)
    17. 7.1 User & Permission Management
    18. 7.2 Module Management
    19. 7.3 Academic Year Management
  20. [Permission Matrix](#)
  21. [Core Business Rules](#)
  22. [Non-Functional Requirements](#)
  23. [Future Scope](#)
  24. [Milestones](#)
-

## 1. Executive Summary

---

### Product Vision

Sirius-Skool is a multi-tenant SaaS School Management System that provides small-to-medium schools with a modular, permission-driven platform to manage their daily operations — from admissions and attendance to results, notices, and reports.

### Target Market

- Private K-12 schools
- Tutoring centers and coaching institutes
- Small school chains (2-10 branches)

### Business Goals

- Enable rapid tenant onboarding (self-service or assisted)
- Provide modular feature access so schools only pay for/use what they need
- Maintain strict data isolation between tenants
- Keep the permission model simple yet powerful — one **Manager** role with granular module + action assignment

### Key Differentiators

- **Modular permission system:** No rigid roles. School Admin creates Managers and assigns specific module+action permissions.
  - **Multi-tenant by design:** Every record is scoped to a tenant from day one.
  - **Action-level granularity:** View, Create, Edit, Delete, Export — per module, per manager.
- 

## 2. Problem Statement

---

Schools currently face one of two extremes:

| Problem                     | Description                                                                                    |
|-----------------------------|------------------------------------------------------------------------------------------------|
| <b>Manual operations</b>    | Excel sheets, paper registers, manual report generation — slow and error-prone                 |
| <b>Over-engineered ERPs</b> | Complex systems designed for large institutions with 20+ roles, unmanageable for small schools |

Neither extreme fits a school that wants: - A simple admin + manager structure - Modular features (not a bloated all-in-one) - Affordable SaaS pricing - Data privacy (their data should not mix with other schools)

## How Sirius-Skool Solves This

| Need           | Solution                                             |
|----------------|------------------------------------------------------|
| Simple roles   | School Admin + Manager (permission-based)            |
| Modularity     | School Admin enables only needed modules             |
| Multi-tenant   | Isolated data per school                             |
| Affordable     | SaaS model, no on-premise infrastructure             |
| Easy reporting | ID cards, TC, attendance/result reports in one click |

---

## 3. Scope

---

### In-Scope (MVP)

| # | Module                          | Priority |
|---|---------------------------------|----------|
| 1 | Authentication                  | P0       |
| 2 | Dashboard                       | P0       |
| 3 | Student Management & Admission  | P0       |
| 4 | Attendance Management           | P0       |
| 5 | Result Management               | P1       |
| 6 | Notice Board                    | P1       |
| 7 | Notification System (SMS/Email) | P1       |
| 8 | Reports                         | P1       |
| 9 | Settings                        | P0       |

### Cross-Cutting (In-Scope)

- User & Permission Management
- Module Management (School Admin enables/disables modules)
- Academic Year Management
- Multi-Tenant Data Isolation
- Audit Logging

### Out-of-Scope (Post-MVP)

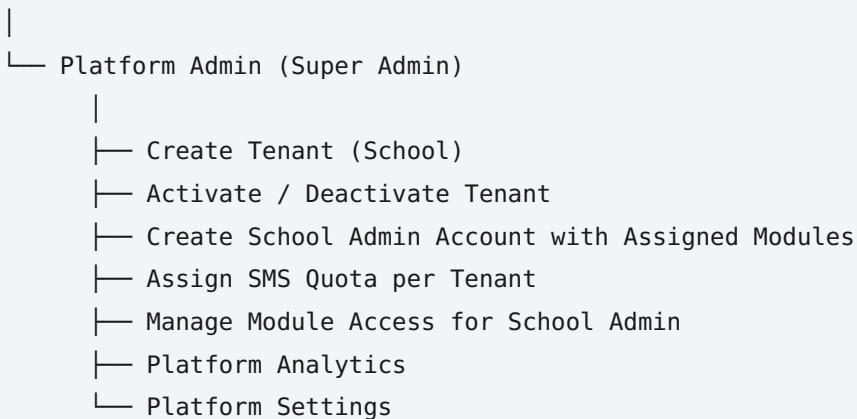
- Fee management / payment gateway

- Parent/student portal
  - Library management
  - Transport management
  - Hostel management
  - Mobile apps (iOS/Android)
  - Online exams / proctoring
  - Payroll / HR management
  - Integration with LMS (Google Classroom, Moodle)
- 

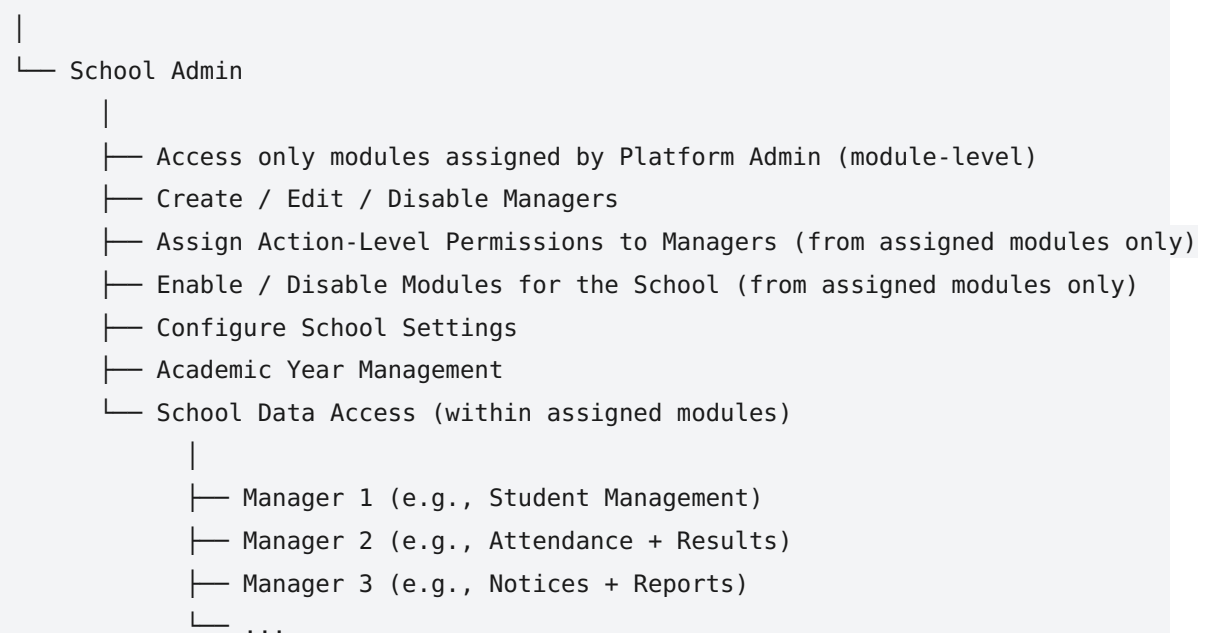
## 4. User Types & Role Hierarchy

---

Platform (SaaS Owner)



Tenant (School)



## Role Definitions

| Role                                | Scope         | Creator        | Access Model                                                                                                                                            |
|-------------------------------------|---------------|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Platform Admin (Super Admin)</b> | All tenants   | System seeded  | Full platform control: create tenants, create School Admin accounts, assign modules at <b>module level</b> , set SMS quotas, platform config            |
| <b>School Admin</b>                 | Single tenant | Platform Admin | <b>Module-level</b> — gets full CRUD + Export on each assigned module. Can then assign <b>action-level</b> permissions to Managers within those modules |
| <b>Manager</b>                      | Single tenant | School Admin   | <b>Action-level</b> — only assigned View / Create / Edit / Delete / Export on specific modules (from the pool available to School Admin)                |

## Key Rules

- There is exactly **one School Admin per tenant**.
  - Platform Admin creates the School Admin account and assigns modules at the **module level** (not action-level). If a module is assigned, School Admin gets all actions on it.
  - School Admin assigns permissions to Managers at the **action level** (e.g., View + Create only, no Delete).
  - School Admin cannot access or assign any module not granted by Platform Admin.
  - School Admin can create any number of Managers and assign them permissions from the available module pool.
- 

## 5. Multi-Tenant Architecture Overview

### Tenant Isolation Model

**Recommended:** Shared database with `tenant_id` column on every table, enforced by middleware.

| Consideration | Shared DB ( <code>tenant_id</code> ) | DB-per-Tenant                     |
|---------------|--------------------------------------|-----------------------------------|
| Cost          | Low (one DB server)                  | High (separate DB per tenant)     |
| Maintenance   | Easy (migrations once)               | Complex (migrations per DB)       |
| Isolation     | Logical (app-enforced)               | Physical (full isolation)         |
| Compliance    | Adequate for most schools            | Required for enterprise/regulated |
| Backup        | Single backup                        | Per-tenant backup                 |

**Recommendation for MVP:** Shared DB with `tenant_id` scoping. Migrate to DB-per-tenant later if enterprise compliance requirements emerge.

## Data Ownership Rules

- Every user record belongs to exactly one tenant.
- Every student record belongs to exactly one tenant.
- Every attendance, result, notice, and report belongs to exactly one tenant.
- Cross-tenant data access is prohibited at the application layer.
- The `tenant_id` is injected into every query via middleware (never trusts user input for tenant scoping).

## Tenant Identity (Subdomain Strategy)

Each tenant is identified by a unique subdomain: - **Format:** `schoolname.sirius-skool.com` - **Platform Admin URL:** `admin.sirius-skool.com` - **Auth flow:** The subdomain identifies the tenant at login. Login page detects the subdomain and scopes authentication to that tenant. - **Wildcard SSL certificate** is required for `*.sirius-skool.com` - **Future:** Custom domains ( `schoolname.com` ) can be offered as a paid upgrade

## Tenant Lifecycle

Create → Activate → (Active) → Deactivate → (Inactive) → Delete (after X days)

- **Create:** Platform Admin provisions a new school with a unique subdomain, selects which modules the School Admin can access, sets the initial SMS quota, configures the starting registration sequence (if migrating from legacy data), and creates the School Admin account.
- **Activate:** School Admin receives credentials via email, logs in, configures school settings, and begins operations within the assigned modules.
- **Deactivate:** Tenant is suspended (data preserved, login blocked, all sessions invalidated).
- **Delete:** Tenant data is purged after a grace period (soft-delete first, hard-delete after X days).

---

## 6. Functional Modules

### 6.1 Authentication

**Purpose:** Secure access to the system with role-based routing.

**Features:** - Login with email/username + password - Logout (invalidate session) - Forgot Password (email-based reset link) - Change Password (authenticated user) - JWT-

based authentication with access + refresh tokens - Session management (single session per user optional) - Role-based redirect after login (School Admin → Admin Dashboard, Manager → Manager Dashboard) - Tenant-scoped login — user enters subdomain or tenant is identified at login

**Business Rules:** - Platform Admin logs in via a separate URL (e.g., `admin.sirius-skool.com`) - School Admin and Managers log in via tenant URL (e.g., `schoolname.sirius-skool.com`) - A user cannot be logged into multiple tenants simultaneously - Inactive/deactivated users cannot log in

---

## 6.2 Dashboard

**Purpose:** Provide role-specific at-a-glance overview of key metrics.

**Features:**

*School Admin Dashboard:* - Total student count (current academic year) - Today's attendance percentage - Recent notices (last 5) - Upcoming exams/events - Manager activity summary - Remaining SMS quota (e.g., "420 / 500 SMS remaining") - Quick actions (Add Student, Take Attendance, Create Notice)

*Manager Dashboard (permissions-dependent):* - Widgets are shown only for assigned modules - If assigned Student Management → show student count - If assigned Attendance → show today's attendance %, pending attendance - If assigned Results → show recent result entries - Quick actions relevant to assigned modules

**Business Rules:** - Dashboard data is scoped to the current academic year - Managers only see data for modules they have View permission on - Metrics are displayed in real-time (no caching for MVP)

---

## 6.3 Student Management & Admission

**Purpose:** Complete student life cycle management within a tenant.

**Features:**

*Student CRUD:* - Add new student (with photo, guardian details, address, etc.) - Edit student details - View student profile (full history) - Delete student (soft delete with restore option) - Search by name, roll number, class, registration number - Filter by class, section, academic year, status (active/transferred/dropped out)

*Admission:*

**Admission Workflow:**

Student fills admission form → Temporary DB (30 days) → Manager reviews → Accept → Main DB  
→ Reject → Deleted from temporary DB

- **Step 1 — Online Form:** Student or parent fills an admission form with all required fields (personal info, guardian details, address, documents). This is stored in a **temporary admission table**.
- **Step 2 — Temp Storage:** Records in the temporary table auto-expire after 30 days (cron job or TTL index).
- **Step 3 — Review Page:** Manager with admission permission sees a paginated list of pending applications. Can search by registration number (assigned on form submission) or filter by date/class.
- **Step 4 — Accept:** Moves the student record from the temporary table to the main `students` table, assigns a permanent registration number, enrolls in the selected class + section, assigns roll number.
- **Step 5 — Reject:** Deletes the record from the temporary table (no trace retained).
- A temporary registration number is generated on form submission for tracking (e.g., `TMP-26000001`). The permanent registration number is assigned only on acceptance.

**Promotion:** - Promote students to next class at academic year end - Bulk promotion with confirmation step - Group promotion (by class) - Roll numbers are auto-assigned on promotion (can be adjusted by School Admin)

**Other Operations:** - Transfer student to another school (with TC generation) - Mark student as dropped out (with date and reason) - Graduate student (completing final year)

**Bulk Operations:** - Import students from Excel (with template download) - Export student list to Excel/PDF

### Registration Number — Format & Generation:

| Attribute       | Detail                                                                                                                     |
|-----------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Format</b>   | YY + Sequence (e.g., <code>26000001</code> )                                                                               |
| <b>YY</b>       | Last two digits of the admission calendar year (e.g., <code>26</code> for 2026, <code>27</code> for 2027)                  |
| <b>Sequence</b> | Continuous, never-resetting integer per tenant, padded to minimum width                                                    |
| <b>Examples</b> | <code>26000001</code> , <code>26000568</code> , <code>27001569</code> , <code>341000000</code> (padded width auto-expands) |

**Generation Mechanism:** - A `tenant_sequences` table stores one counter per tenant ( `registration_sequence` ). - On admission: read counter → generate `YY + padded(sequence)` → increment counter → commit (all in one atomic transaction). - The sequence starts at 1 for new tenants. For schools migrating from legacy data, the starting value is configurable at tenant setup.

### Roll Number:

| Attribute          | Detail                                                                        |
|--------------------|-------------------------------------------------------------------------------|
| <b>Purpose</b>     | Class + section daily identifier                                              |
| <b>Scope</b>       | Unique per (class + section + academic year)                                  |
| <b>Reset</b>       | Re-assigned each academic year on promotion                                   |
| <b>Storage</b>     | <code>student_enrollments.roll_number</code> — not in the registration number |
| <b>Auto-assign</b> | Sequential within section (1-35 for section A, 1-32 for section B)            |
| <b>Override</b>    | School Admin can manually adjust individual roll numbers                      |

**Section:** - A section is a subgroup within a class (e.g., Class 7 → Sections A, B, C). - Sections are created by School Admin in Settings. - A student is enrolled in one class + one section per academic year. - A student can change section in a new academic year or on promotion.

#### Data Model (conceptual):

```

classes (id, name, tenant_id)
sections (id, class_id, name, tenant_id)

student_enrollments (
  student_id, class_id, section_id,
  academic_year_id, roll_number,
  tenant_id
)

```

**Business Rules:** - Registration number is auto-generated, unique per tenant (enforced by `UNIQUE(tenant_id, registration_number)`), and never changes. - The class/section is NOT encoded in the registration number — students can be promoted without their permanent ID becoming inaccurate. - Roll number is distinct from registration number and exists only on the enrollment record. - A student can have multiple enrollments across academic years, but only one active enrollment at a time. - Deleting a student requires confirmation and is soft-delete (data preserved for reports). - Promoting a student creates a new enrollment in the next class for the new academic year; the previous year's enrollment becomes historical.

## 6.4 Attendance Management

**Purpose:** Record and track daily student attendance.

#### Features:

**Attendance UI Flow:** 1. Manager/Admin selects **Class** → **Section** → **Date** 2. System loads all enrolled students for that class + section, displaying **Name** and **Roll Number** 3. All students are **checked as Present by default** 4. Manager unchecks any student

who is **Absent** 5. After marking, Manager clicks "**Save Attendance**" 6. A separate "**Send SMS**" button appears — clicking it sends an absence notification SMS to the guardians of all students marked absent

**Other Features:** - Edit attendance within a configurable window (e.g., same day only) - View attendance history — per class, per student, per date range - Monthly attendance report (tabular + summary) - Per-student attendance report (percentage calculation)

**Business Rules:** - Attendance is scoped to academic year + class + section - All students default to Present — only absentees are explicitly marked - Only today's and future dates can have attendance taken (no back-dating beyond 7 days, configurable) - Edited attendance is logged (who changed what, when) - Once a month is closed (configured by School Admin), attendance becomes read-only - "Send SMS" sends only to guardians of absent students (uses system's centralized SMS gateway) - Each SMS sent consumes from the tenant's SMS quota (see Notification System)

---

## 6.5 Result Management

**Purpose:** Create exams, record marks, and publish results.

**Features:** - Create exam types (Midterm, Final, Quiz, etc.) - Configure subjects per class - Enter marks per student per subject (numeric or grade-based) - Auto-calculate total, percentage, rank - Publish/unpublish results - View result sheet (tabular — student-wise or class-wise) - Generate rank list - Automatic SMS/Email notification to parents when results are published

**Business Rules:** - Results are scoped to academic year + exam + class - Marks can be entered only for the current academic year - Once marks are submitted and published, editing requires "unpublish" first - Published results are visible in reports and student history - Marks entry can be done by multiple managers (e.g., subject-wise) if permissions allow

---

## 6.6 Notice Board

**Purpose:** Publish and manage notices for staff and students.

**Features:** - Create notice (title, body, attachments) - Publish notice immediately - Schedule notice for future date (auto-publish) - Archive notice (hide from active view, keep for records) - Delete notice (soft delete) - Filter by date range, status (published/scheduled/archived) - Notices are visible on the dashboard

**Business Rules:** - Notices are tenant-scoped - Scheduled notices auto-publish at the configured date/time - Published notices cannot be edited (must archive and recreate) - Archived notices are retained for audit/report purposes

---

## 6.7 Notification System

**Purpose:** Send automated and manual notifications via SMS and Email. The platform provides centralized notification infrastructure — tenants do not configure their own gateways.

**Architecture:** Centralized Gateway - The platform (Sirius-Skool) negotiates a bulk rate with SMS (e.g., Twilio) and Email (e.g., SendGrid) providers. - All tenant notifications are sent through the platform's centralized gateway. - Usage is tracked per tenant for billing/monitoring. - This provides a seamless experience — School Admins never configure SMTP or SMS API keys.

**SMS Quota System:** - Each tenant has a **SMS quota** (credits) assigned by Platform Admin. - Platform Admin sets and adjusts the quota from their dashboard (e.g., 500 SMS/month). - Every SMS sent by the tenant consumes one credit from their quota. - When quota is exhausted, SMS sending is blocked (Email still works if configured). - School Admin can view their remaining SMS balance on the dashboard. - A warning notification is sent to School Admin when balance is low (e.g., < 10% remaining).

**Features:** - **SMS:** - Attendance notification (absent student) - Result notification (marks published) - Custom SMS (ad-hoc, sent by School Admin or authorized Manager) - **Email:** - Attendance notification - Result notification - Forgot Password / Reset Password (platform-level) - Custom email (ad-hoc) - Notification logs (view sent/failed status per notification with tenant context) - Retry failed notifications (configurable retry policy per tenant)

**Notification Templates:** - Default templates are provided by the platform for each notification type. - School Admin can customize templates (edit the message body while preserving variables). - Template variables: {student\_name}, {class}, {section}, {date}, {school\_name}, {percentage} (results), {subject} (results) - Customizations are per-tenant and do not affect other tenants.

**Business Rules:** - Notification failures must not roll back the triggering action (e.g., attendance mark or result publish succeeds even if SMS fails) - Platform Admin manages the central gateway credentials (not exposed to tenants) - Platform Admin sets and adjusts SMS quota per tenant from the platform dashboard - Each SMS sent consumes one credit from the tenant's quota; sending is blocked when quota is 0 - Notification logs are retained per tenant for audit - Platform Admin can view notification logs across all tenants for support - Template changes apply to the next notification, not retroactively

---

## 6.8 Reports

**Purpose:** Generate and export documents for students.

**Features:**

*Student Reports:* - ID Card (with photo, name, class, roll number, school details) - Admission Form (filled admission details) - Student Profile (complete history)

*Certificates:* - Transfer Certificate (TC) — generated on student transfer - Testimonial / Character Certificate

*Academic Reports:* - Attendance Report (per student or per class) - Result Report (per exam or consolidated)

*Export:* - PDF (all reports) - Excel (attendance/result data)

**Business Rules:** - All reports include school branding (logo, name, address from Settings) - Reports are generated for the current academic year by default, with option to select previous years - Data is read-only at report generation time (reports never modify data) - Reports include only data belonging to the current tenant - ID card and certificate templates are consistent across the tenant (same layout, auto-filled with school details)

---

## 6.9 Settings

**Purpose:** Tenant-level configuration managed by School Admin.

**Features:**

*School Information:* - School name, logo, address, phone, email - Website URL - School code/registration number

*Academic:* - Academic Year management (create, set active, close) - Class/Section setup (create classes, sections within classes) - Subject setup (per class)

*Notifications:* - Notification template customization (edit SMS and Email message bodies per notification type) - Enable/disable notification types (e.g., disable SMS attendance alerts, keep email) - Sender name/ID override (customize the "from" name shown to recipients)

*Security:* - Password policy (minimum length, complexity requirements) - Session timeout duration

*General:* - Language preference (for future i18n) - Timezone, date format, number format

*Registration:* - Set starting registration sequence value (for schools migrating from legacy systems)

**Business Rules:** - Settings are per-tenant, never shared - Only School Admin can modify settings - Notification template changes apply to the next notification, not retroactively - Academic Year can only be closed if all attendance and results for that year are finalized - Gateway credentials are managed by Platform Admin (centralized), not exposed in tenant Settings

---

## 7. Cross-Cutting Modules

---

These are not separate menu items but affect every module and feature in the system.

## 7.1 User & Permission Management

**Purpose:** School Admin creates and manages Manager accounts with action-level permissions (limited to modules assigned by Platform Admin at module-level).

### Access Model (Two-Tier):

| Level         | Assigner → Assignee           | Granularity                                                                                                                        |
|---------------|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tier 1</b> | Platform Admin → School Admin | <b>Module-level</b> — if assigned, School Admin gets full access (View, Create, Edit, Delete, Export). No action-level restriction |
| <b>Tier 2</b> | School Admin → Manager        | <b>Action-level</b> — School Admin picks which specific actions the Manager gets (e.g., View + Create only)                        |

**Features:** - Create Manager (name, email, password) - Edit Manager details - Activate / Deactivate Manager - Assign **action-level** permissions to Manager (View, Create, Edit, Delete, Export per module) - Only modules assigned by Platform Admin appear in the list for School Admin to delegate - View list of all Managers with their permission summary

### Permission Model (Tier 2 — School Admin → Manager):

Module: Student Management

- └ View
- └ Create
- └ Edit
- └ Delete
- └ Export

Module: Attendance

- └ View
- └ Take (Create)
- └ Edit

Module: Results

- └ View
- └ Enter Marks (Create)
- └ Edit Marks
- └ Publish

Module: Notice Board

- └ View
- └ Create
- └ Edit
- └ Delete
- └ Schedule

Module: Reports

- └ View
- └ Export

Module: Settings

- └ (School Admin only – not assignable)

Module: User Management

- └ (School Admin only – not assignable)

**Business Rules:** - Platform Admin assigns modules at **module-level** — School Admin gets all actions on an assigned module - School Admin can only delegate modules that Platform Admin assigned — they cannot delegate what they don't have - School Admin assigns permissions to Managers at **action-level** — selecting which specific operations each Manager can perform - A Manager with no permissions cannot see any module or data - Permissions are additive — granting View + Create means the Manager can both see and create records - Deactivating a Manager immediately revokes access (active sessions are invalidated)

---

## 7.2 Module Management

**Purpose:** School Admin can enable or disable modules that Platform Admin has assigned to them.

**Features:** - View all modules assigned by Platform Admin with individual toggle on/off - When a module is disabled, it is hidden from the sidebar and inaccessible to all users (including School Admin) - Disabling a module does not delete associated data - Modules not assigned by Platform Admin are not visible in this list

**Business Rules:** - Authentication and Settings modules cannot be disabled - School Admin cannot enable a module that Platform Admin did not assign - Disabling a module hides it for the entire tenant - Re-enabling a module restores access to existing data

---

## 7.3 Academic Year Management

**Purpose:** Every transaction is scoped to an academic year. Schools manage their own academic years.

**Features:** - Create Academic Year (name, start date, end date) - Set one Academic Year as active - Close Academic Year (make read-only) - Re-open closed year (if needed)

**Business Rules:** - Exactly one Academic Year can be active per tenant at any time - All data entry (attendance, results, admissions) happens in the active academic year - Previous academic years become read-only after closure — no edits allowed, only

viewing reports - Students are promoted by creating a new enrollment in the next academic year - Academic years cannot overlap in date ranges within a tenant

## 8. Permission Matrix

**Two-Tier Model:** - Tier 1 (Platform Admin → School Admin): **Module-level** — if a module is assigned, School Admin gets *all* actions on it. No action-level restriction. - Tier 2 (School Admin → Manager): **Action-level** — School Admin selects specific actions for each Manager.

| Module                | Action          | Platform Admin | School Admin        | Manager                     |
|-----------------------|-----------------|----------------|---------------------|-----------------------------|
| <b>Authentication</b> | Login           |                |                     |                             |
|                       | Logout          |                |                     |                             |
|                       | Change Password |                |                     |                             |
| <b>Dashboard</b>      | View            |                |                     | (permissions-dependent)     |
| <b>Student Mgmt</b>   | All actions     |                | if module assigned* | As assigned by School Admin |
| <b>Attendance</b>     | All actions     |                | if module assigned* | As assigned by School Admin |
| <b>Results</b>        | All actions     |                | if module assigned* | As assigned by School Admin |
| <b>Notice Board</b>   | All actions     |                | if module assigned* | As assigned by School Admin |
| <b>Notifications</b>  | All actions     |                | if module assigned* | As assigned by School Admin |
| <b>Reports</b>        | All actions     |                | if module assigned* | As assigned by School Admin |
| <b>Settings</b>       | All actions     |                | if module assigned* |                             |
| <b>User Mgmt</b>      | All             |                | (always)            |                             |
| <b>Module Mgmt</b>    | All             |                | (always)            |                             |
| <b>Academic Year</b>  | All             |                | (always)            |                             |
| <b>Platform</b>       | All             |                |                     |                             |

- = Always granted

- = Never granted
  - **if module assigned\*** = School Admin gets this module (with all its actions) only if Platform Admin assigned it at module-level. If assigned, full CRUD + Export included.
  - **As assigned by School Admin** = Manager gets only the specific actions (View, Create, Edit, Delete, Export) the School Admin chooses to delegate within the available module pool.
- 

## 9. Core Business Rules

---

### Tenant Rules

1. Every user, student, and record belongs to exactly one tenant.
2. Cross-tenant data access is prohibited at every layer (API, DB, UI).
3. Tenant deactivation immediately blocks all user access for that tenant.
4. Platform Admin cannot directly modify tenant operational data (students, attendance, results) — only support/view.

### Academic Rules

1. Only one Academic Year can be active per tenant at any time.
2. Students are promoted by creating a new enrollment in the next Academic Year.
3. Previous Academic Years become read-only after closure.
4. All attendance, result, and admission operations happen in the active Academic Year.

### Permission Rules (Two-Tier Model)

1. **Tier 1 — Platform Admin → School Admin:** Platform Admin assigns modules at **module-level**. If a module is assigned, School Admin gets all actions (View, Create, Edit, Delete, Export) on that module. No action-level restriction.
2. **Tier 2 — School Admin → Manager:** School Admin assigns permissions at **action-level** within the modules they received from Platform Admin. They can give a Manager View-only, Create-only, or any combination.
3. School Admin can only delegate modules that Platform Admin granted — they cannot delegate what they don't have.
4. A Manager with no assigned permissions cannot see any module or data.
5. Permissions are additive — granting View + Create means the Manager can both see and create records.
6. Deactivating a Manager immediately revokes access across all sessions.

## SMS Quota Rules

1. Each tenant has an SMS quota assigned by Platform Admin.
2. Every SMS sent consumes one credit from the tenant's quota.
3. SMS sending is blocked when the quota reaches zero (email still works).
4. Platform Admin can adjust the quota for any tenant from the platform dashboard at any time.

## Registration Number Rules

1. Registration number format is `YY + continuous_sequence` (e.g., `26000001`). No class, section, or other changeable attribute is embedded.
2. The sequence is stored per tenant in a `tenant_sequences` table and increments atomically on each admission.
3. Registration number is unique per tenant ( `UNIQUE(tenant_id, registration_number)` ) and never changes for the student's entire tenure.
4. Starting sequence value is configurable at tenant creation for legacy migrations.

## Data Rules

1. Deleting a record performs a soft delete (data preserved for reports/audit).
2. Student, attendance, and result data cannot be permanently deleted once created (only soft-deleted).
3. Audit logs are immutable — once written, they cannot be edited or deleted.

## Notification Rules

1. Notification failures must not roll back the source transaction (attendance, result publish, etc.).
2. All notifications are sent through the platform's centralized gateway (no per-tenant gateway configuration).
3. Notification logs are retained for troubleshooting.

## Reporting Rules

1. Reports are generated within the selected Academic Year.
  2. Reports include only data belonging to the current tenant.
  3. All reports include school branding from Settings.
-

## 10. Non-Functional Requirements

### Performance

**Why flexible targets?** These are initial targets for MVP and will be refined through real-world usage. Actual performance depends on deployment infrastructure (server spec, DB size, network). The architecture is designed to scale horizontally, so these targets can be met by adding resources rather than rewriting code.

| Metric                      | Target (MVP)        | Notes                                                                                                                  |
|-----------------------------|---------------------|------------------------------------------------------------------------------------------------------------------------|
| Page load time              | < 2-3 seconds (P95) | Target for initial deployment. Can be improved with caching (Redis) and CDN as traffic grows                           |
| API response time           | < 500ms-1s (P95)    | Most CRUD endpoints should respond faster; report generation may take longer                                           |
| Concurrent users per tenant | 50-100+             | Depends on school size. Horizontal scaling can increase this linearly                                                  |
| Total tenants (MVP)         | 100+                | Shared DB with tenant_id scales well to hundreds of tenants. Monitor query performance and add read replicas if needed |

### Security

| Requirement      | Detail                                                                            |
|------------------|-----------------------------------------------------------------------------------|
| Authentication   | JWT (access + refresh tokens)                                                     |
| Password storage | bcrypt or Argon2                                                                  |
| Data encryption  | TLS 1.3 in transit, AES-256 at rest                                               |
| API security     | Rate limiting, CORS, CSRF protection                                              |
| Input validation | All inputs sanitized and validated server-side                                    |
| Audit logging    | All create/update/delete operations logged with user ID, tenant ID, timestamp, IP |

### Scalability

- Horizontal scaling: application layer can scale independently
- Database: read replicas for reporting queries (future)

- Caching: Redis for session store and frequently accessed data (classes, settings)
- **File storage:** Cloudinary for all file uploads — student photos, ID card templates, notice attachments, report exports
- Benefits: CDN delivery, automatic image optimization, transformation (resize/crop for ID cards), secure upload with signatures

## Availability

| Metric             | Target                                   |
|--------------------|------------------------------------------|
| Uptime SLA         | 99.5% (MVP), 99.9% (post-MVP)            |
| Maintenance window | Off-peak hours with notice               |
| Backup             | Daily automated backup, 30-day retention |

## Data Isolation

- All queries are scoped by `tenant_id` via application middleware
- No raw SQL queries bypass tenant scoping
- Export/import operations are also tenant-scoped

## Internationalization (Future)

- Database schema should support multi-language labels (consider using locale tables from day one)
- Date, time, and number formats should be configurable per tenant

# 11. Future Scope

These features are intentionally excluded from MVP to keep scope focused and deliverable. They are noted here to inform architectural decisions.

| Module                | Priority | Notes                                                                              |
|-----------------------|----------|------------------------------------------------------------------------------------|
| Fee Management        | High     | Fee structure, invoices, payment gateway (Stripe/SSLCommerz), due reminders        |
| Parent/Student Portal | High     | Separate login for parents/students to view attendance, results, notices, pay fees |
| Library Management    | Medium   | Book catalog, issue/return, fines, barcode scanning                                |
| Transport Management  | Medium   | Routes, GPS tracking, stops, fee calculation, vehicle maintenance                  |
|                       | Medium   |                                                                                    |

| Module            | Priority | Notes                                                         |
|-------------------|----------|---------------------------------------------------------------|
| Hostel Management |          | Room allocation, attendance, fee management, warden dashboard |
| Payroll / HR      | Low      | Staff salary, leave management, attendance for staff          |
| Online Exams      | Low      | Timed online tests, auto-grading, proctoring                  |
| Mobile Apps       | Low      | iOS/Android apps for parents, teachers, students              |
| LMS Integration   | Low      | Google Classroom, Moodle sync                                 |
| Multi-Language    | Medium   | i18n support for UI and reports                               |

**Architectural note for future scope:** The permission system (module + action) should be designed to easily accommodate new modules without code changes. A new module like "Fee Management" should be auto-discoverable and assignable via the same permission UI.

---

## 12. Milestones

---

### Phase 1 — Foundation (MVP)

| Milestone | Modules                                          | Duration (est.) |
|-----------|--------------------------------------------------|-----------------|
| M1.1      | Tenant onboarding, Auth, Settings, Academic Year | 3-4 weeks       |
| M1.2      | User & Permission Management, Module Management  | 2 weeks         |
| M1.3      | Student Management & Admission                   | 3-4 weeks       |
| M1.4      | Attendance Management                            | 2 weeks         |
| M1.5      | Result Management                                | 2-3 weeks       |
| M1.6      | Notice Board, Notifications, Reports             | 3-4 weeks       |
| M1.7      | Dashboard, Audit Logs, Polish, Deployment        | 2 weeks         |

**Total MVP: ~16-20 weeks (4-5 months)**

### Phase 2 — Enhancement

| Feature                | Goal                                   |
|------------------------|----------------------------------------|
| Fee Management         | Revenue operations                     |
| Parent/Student Portal  | Reduce parent workload on school staff |
| Multi-Language Support | Expand to non-English markets          |

## Phase 3 — Scale

| Feature                | Goal                             |
|------------------------|----------------------------------|
| Mobile Apps            | Increase accessibility           |
| Online Exams           | Expand use case                  |
| LMS Integration        | Compete with larger ERPs         |
| Analytics & BI         | Data-driven insights for schools |
| Subscription & Billing | Automate tenant payments         |

---

*End of PRD*